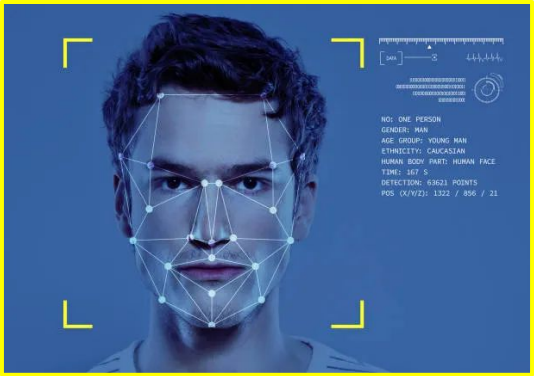
Python Facial Recognition App using a Siamese Neural Network

Max Weinstein

Facial Recognition

- **Facial verification:** does this person's face belong to the person in my reference database?
- **Facial identification:** does this person's face match any of the faces in my reference database?
- **Facial attribute classification:** can we analyze the attributes of a person's face to estimate their age or gender?
- **Facial affect recognition:** can we analyze someone's facial expressions in real-time to determine their emotions?



Facial Recognition

- For this project, I focused on **facial verification**, trying to mimic the functionality of Apple's FaceID but instead through a Kivy application, Python's platform for GUI app development.

“Siamese Neural Networks for One-shot Image Recognition”

Siamese Neural Networks for One-shot Image Recognition

Gregory Koch GKOCH@CS.TORONTO.EDU
Richard Zemel ZEMEL@CS.TORONTO.EDU
Ruslan Salakhutdinov RSALAKHU@CS.TORONTO.EDU
Department of Computer Science, University of Toronto. Toronto, Ontario, Canada.

Abstract

The process of learning good features for machine learning applications can be very computationally expensive and may prove difficult in cases where little data is available. A prototypical example of this is the *one-shot learning* setting, in which we must correctly make predictions given only a single example of each new class. In this paper, we explore a method for learning *siamese neural networks* which employ a unique structure to naturally rank similarity between inputs. Once a network has been tuned, we can then capitalize on powerful discriminative features to generalize the predictive power of the network not just to new data, but to entirely new classes from unknown distributions. Using a convolutional architecture, we are able to achieve strong results which exceed those of other deep learning models with near state-of-the-art performance on one-shot classification tasks.

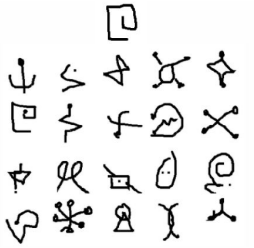


Figure 1. Example of a 20-way one-shot classification task using the Omniglot dataset. The lone test image is shown above the grid of 20 images representing the possible unseen classes that we can choose for the test image. These 20 images are our only known examples of each of those classes.

- Traditional classifiers require retraining every time you add a new class (add output nodes).
- A Siamese Neural Network can avoid the pain of having to retrain the entire network each time a person’s face is added by learning the similarity between images as opposed to classifying each image as a specific output.
- Uses of images of faces as opposed to images of digits like the paper.

Siamese Neural Network Architecture

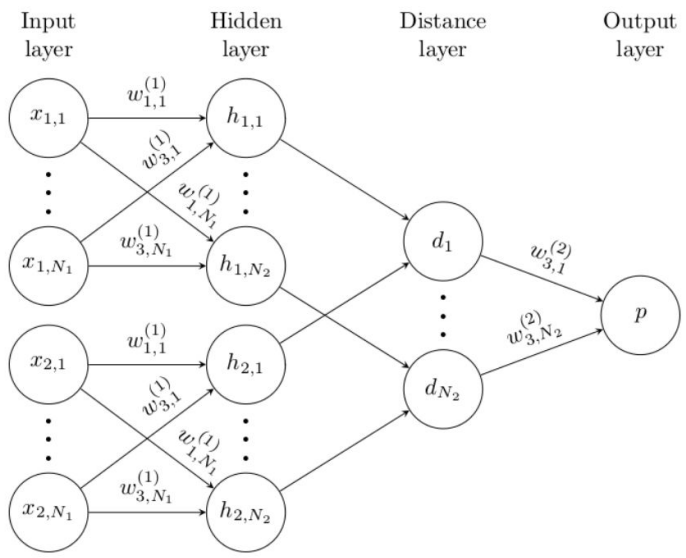


Figure 3. A simple 2 hidden layer siamese network for binary classification with logistic prediction p. The structure of the network is replicated across the top and bottom sections to form twin networks, with shared weight matrices at each layer.

- Pass in two images at same time: one from the person's webcam, the other from a reference dataset
- Hidden layer turns converts the images to feature vectors (compressed representation of facial features)

Siamese Neural Network Architecture

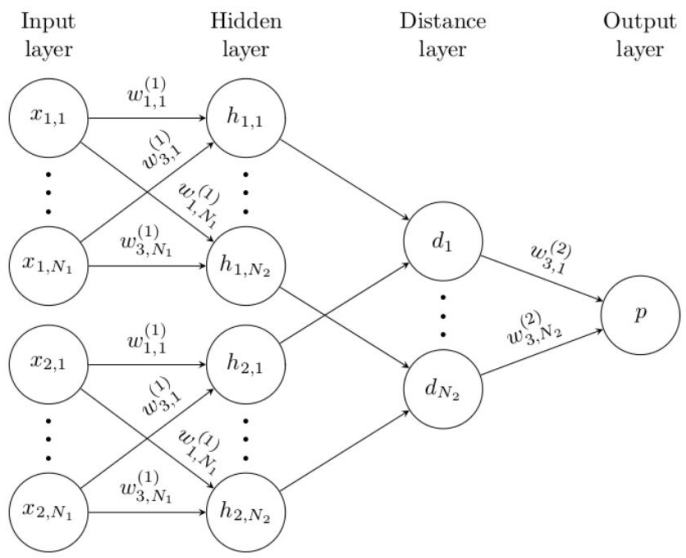


Figure 3. A simple 2 hidden layer siamese network for binary classification with logistic prediction p. The structure of the network is replicated across the top and bottom sections to form twin networks, with shared weight matrices at each layer.

- Distance layer measures the similarity by calculating how far apart they are using L1 distance (output of 1 = similar)
- Both networks share the same weights (same network applied twice) so that if two faces are similar, we'd expect both networks to similar feature vectors

Siamese Neural Network Architecture

- Pass two images through the network
- Each image will undergo a series of convolutions (collect facial features) and max pooling (retain most important facial features) before being converted into a 4096 unit feature embedding
- Calculate similarity via the L1 distance between the feature vectors

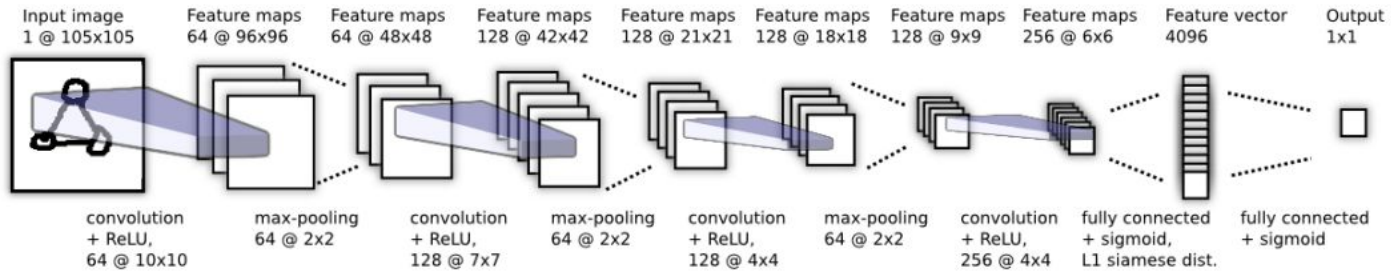


Figure 4. Best convolutional architecture selected for verification task. Siamese twin is not depicted, but joins immediately after the 4096 unit fully-connected layer where the L1 component-wise distance between vectors is computed.

Building the Siamese Neural Network (SNN)

Setup

- Pass in two of the three categories into the network at a time:
  - Anchor - real-time input image (webcam using OpenCV)
  - Positive - positive verification image dataset (webcam using OpenCV)
  - Negative - negative verification image (Labelled Faces in the Wild)

```
# Setup paths
# os.path.join joins different directories together to form full file path
# Setup paths
POS_PATH = os.path.join('data', 'positive')
NEG_PATH = os.path.join('data', 'negative')
ANC_PATH = os.path.join('data', 'anchor')
```

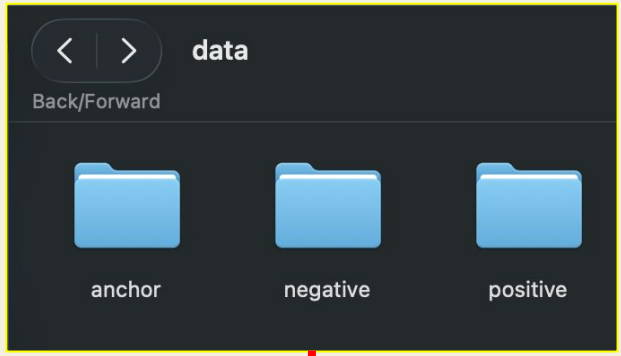
```
# Make the directories
os.makedirs(POS_PATH, exist_ok=True)
os.makedirs(NEG_PATH, exist_ok=True)
os.makedirs(ANC_PATH, exist_ok=True)
```

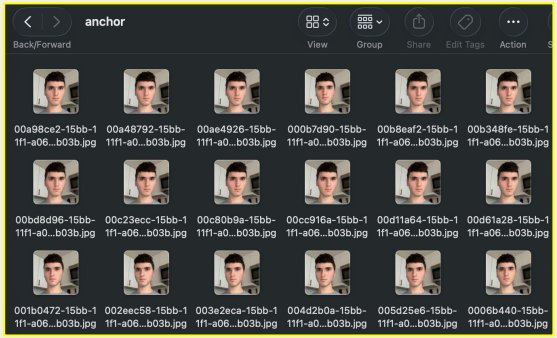
Collecting Positives and Anchors

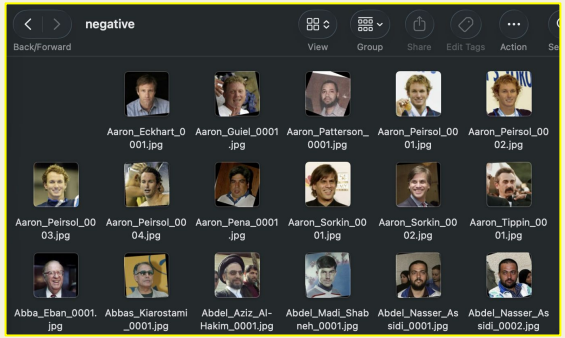
```python
# Establish a connection to the webcam
cap = cv2.VideoCapture(0)

# Counter for number of images saved
anchor_count = 0
positive_count = 0

# Loop through every frame in webcam
while cap.isOpened():
    ret, frame = cap.read()

    # Cut down frame to 950x950px
    frame = frame[150:1100, 550:1500, :]

    # Take and save 300 anchor images
    if anchor_count < 300:
        imgname = os.path.join(ANC_PATH, '{}.jpg'.format(uuid.uuid1()))
        cv2.imwrite(imgname, frame)
        anchor_count += 1
    else:
        break

    # Show image back to screen
    cv2.imshow('Image Collection', frame)

    # Wait 1mS between frames
    cv2.waitKey(1)

# Release the webcam
cap.release()
cv2.destroyAllWindows()
cv2.waitKey(1)
```

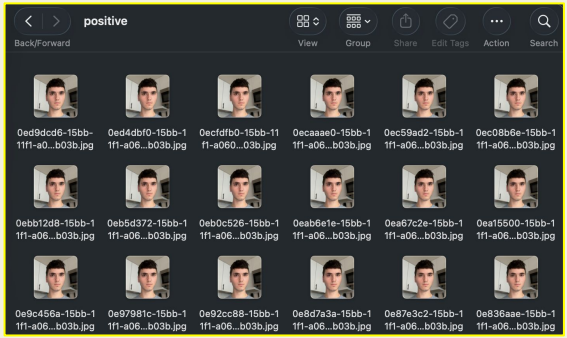
```python
# Establish a connection to the webcam
cap = cv2.VideoCapture(0)

# Counter for number of images saved
anchor_count = 0
positive_count = 0

# Loop through every frame in webcam
while cap.isOpened():
    ret, frame = cap.read()

    # Cut down frame to 950x950px
    frame = frame[150:1100, 550:1500, :]

    # Take and save 300 positive images
    if positive_count < 300:
        imgname = os.path.join(POS_PATH, '{}.jpg'.format(uuid.uuid1()))
        cv2.imwrite(imgname, frame)
        positive_count += 1
    else:
        break

    # Show image back to screen
    cv2.imshow('Image Collection', frame)

    # Wait 1mS between frames
    cv2.waitKey(1)

# Release the webcam
cap.release()
cv2.destroyAllWindows()
cv2.waitKey(1)
```

Collecting Positives and Anchors









Loading and Preprocessing Images

- Resizing images to be 100x100
- Convert pixel RGB values from 0-255 to 0-1 (makes calculating gradients much easier)

```python
def preprocess(file_path):
    # Read in image from file path
    byte_img = tf.io.read_file(file_path)
    # Load the image
    img = tf.io.decode_jpeg(byte_img)
    # Preprocessing steps - resizing the image to be 100x100x3
    img = tf.image.resize(img, (100, 100))
    # Scale image to be between 0 and 1
    img = img / 255.0
    # Return image
    return img
```

Making Labelled Dataset

- Making a dataset of labeled pairs
  - Bundles anchor and positive images with label 1
  - Bundles anchor and negative images with label 0
- Model will use this labelled dataset during training to calculate loss

```
# Zips together every anchor image with a positive image and puts a label of 1 on each pair
positives = tf.data.Dataset.zip((anchor, positive, tf.data.Dataset.from_tensor_slices(tf.ones(len(anchor)))))
# Zips together every anchor image with a negative image and puts a label of 0 on each pair
negatives = tf.data.Dataset.zip((anchor, negative, tf.data.Dataset.from_tensor_slices(tf.zeros(len(anchor)))))
# Combine both lists into one big dataset
data = positives.concatenate(negatives)
```

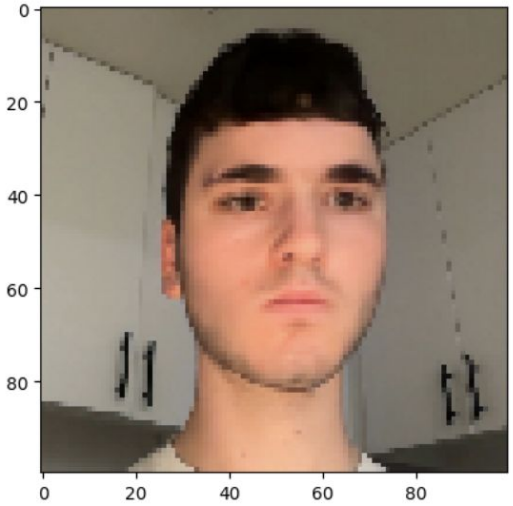
```
# Iterator to step through combined dataset
samples = data.as_numpy_iterator()
# Prints item we're pointing at (sanity check)
samples.next()
```

```
(b'data/anchor/31b4398e-1432-11f1-a0c6-8a8a088db03b.jpg',
 b'data/positive/960ec806-14c1-11f1-a0c6-8a8a088db03b.jpg',
 np.float32(1.0))
```

Making Labelled Dataset

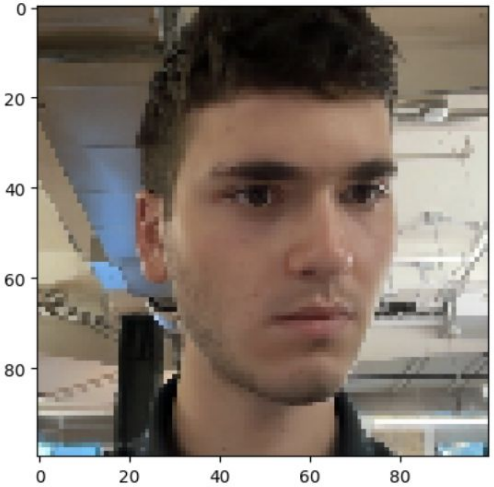
```
[26]: plt.imshow(res[0])
```

```
[26]: <matplotlib.image.AxesImage at 0x124a00b90>
```




```
[27]: # Second image (positive or negative) after preprocessing
plt.imshow(res[1])
```

```
[27]: <matplotlib.image.AxesImage at 0x124a65a90>
```




```
# Label value from first pair (1.0 = same person, 0.0 = different person)
res[2]
```

```
np.float32(1.0)
```

Training and Testing Data

```python
# Build dataloader pipeline
# Running preprocess twin function which returns preprocessed image for anchor, for validation, and label
data = data.map(preprocess_twin)
# Caching images (save preprocessed images in memory so we don't have to reprocess them every time)
data = data.cache()
# Shuffle up images so model doesn't learn order - forces it to actually learn face differences
data = data.shuffle(buffer_size=10000)
```

```python
# Training partition
# Grabbing first 70% of our dataset as training partition
train_data = data.take(round(len(data)*.7))
# Pass in our images in groups of 16 - model processes 16 at a time
train_data = train_data.batch(16)
# Start preprocessing next set of images while we're training on current batch - prevents bottleneck
train_data = train_data.prefetch(8)
```

```python
# Testing partition
# Skip first 70 percent of dataset so we don't pick up training observations
test_data = data.skip(round(len(data)*.7))
# Allocated last 30 percent for test data
test_data = test_data.take(round(len(data)*.3))
test_data = test_data.batch(16)
test_data = test_data.prefetch(8)
```

```python
# Size of training data (should be 420)
round(len(data)*.7)
```

```
420
```

```python
# Size of test data (should be 180)
round(len(data)*.3)
```

```
180
```

- Randomize order of pairs so the model prioritizes learning facial features over sequence of images
- 70% training data
- 30% test data
- Batch size of 16

Model Architecture

```python
# Blueprint for our neural network
def make_embedding():
    # Input layer - accepts 100x100 image with 3 colour channels (RGB)
    inp = Input(shape=(100, 100, 3), name='input_image')

    # First block
    # Convolution composed of 64 filters with 10x10 shape and ReLu activation
    c1 = Conv2D(64, (10, 10), activation='relu')(inp)
    # Every MaxPooling is applied, image gets smaller because 2x2 window can't hang off edges
    # - only works where it has a full 2x2 area. Over many layers, this means that there is size
    # loss. By setting padding to same, it adds a border of zeros around the image before undergoing
    # max pooling so now the 2x2 window has enough room to cover entire image which keeps the output
    # size consistent throughout the network.
    m1 = MaxPooling2D(64, (2, 2), padding='same')(c1)

    # Second block
    # Convolution of 128 filters with 7x7 shape + ReLu activation and passing through results of MaxPooling layer
    c2 = Conv2D(128, (7, 7), activation='relu')(m1)
    # Max pooling of 64 units with 2x2 shape, padding same for consistent sizing
    m2 = MaxPooling2D(64, (2, 2), padding='same')(c2)

    # Third block
    # Convolution of 128 filters with 4x4 shape, ReLu activation, m2 as input to c3
    c3 = Conv2D(128, (4, 4), activation='relu')(m2)
    # Max Pooling of 64 units with 2x2 shape, same padding, and input c3 layer
    m3 = MaxPooling2D(64, (2, 2), padding='same')(c3)

    # Final embedding block
    # Convolution of 256 filters with 4x4 shape, ReLu activation, and m3 layer as input
    c4 = Conv2D(256, (4, 4), activation='relu')(m3)
    # Flattening convolution layer from 3D into 1D
    f1 = Flatten()(c4)
    # Compresses flattened numbers into list of 4096 numbers that depict unique "fingerprint" of face
    # Sigmoid squishes all values between 0 and 1
    # This 4096 vector is what gets compared between two images to see if they are the same person
    # Measure distance between the two vectors - small distance means same person
    d1 = Dense(4096, activation='sigmoid')(f1)

    # Return final embedding model
    return Model(inputs=[inp], outputs=[d1], name='embedding')
```

- Input is 100x100x3
- 4 convolution & max pooling blocks
- Finish with dense layer to calculate element-wise distance between the two 4096 vectors

Model Architecture

- Passing in anchor and validation image into network
- Call embedding on both images to calculate L1 element-wise absolute difference between vectors
- Takes resulting 4096 element vector and compresses into classifier number

```python
def make_siamese_model():

    # Anchor image in the network
    input_image = Input(name='input_img', shape=(100,100,3))

    # Validation image in the network
    validation_image = Input(name='validation_img', shape=(100,100,3))

    # Combine siamese distance components
    siamese_layer = L1Dist()
    siamese_layer._name = 'distance'
    distances = siamese_layer(embedding(input_image), embedding(validation_image))

    # Classification layer
    # Passing 4096 values in which depict difference between the 4096 values of the input embedding
    # and the 4096 avlues of the validation embedding respectively
    classifier = Dense(1, activation='sigmoid')(distances)

    # Return base Model class with inputs as image and validation image and output as classifier
    # Both streams running simultaneously, combine together on L1 distance layer, and output classification layer
    return Model(inputs=[input_image, validation_image], outputs=classifier, name='SiameseNetwork')
```

Training Step

```python
def train(data, EPOCHS):
    # Lists to store metrics across epochs for plotting
    precision_history = []
    recall_history = []

    # Loop through epochs
    for epoch in range(1, EPOCHS+1):
        print('\n Epoch {}/{}'.format(epoch, EPOCHS))
        progbar = tf.keras.utils.Progbar(len(data))

        # Reset metrics at the start of each epoch
        r = Recall()
        p = Precision()

        # Loop through each batch
        for idx, batch in enumerate(data):
            # Run train step here
            loss = train_step(batch)
            # Make predictions on current batch
            yhat = siamese_model.predict(batch[:2])
            # Update precision and recall with current batch results
            r.update_state(batch[2], yhat)
            p.update_state(batch[2], yhat)
            progbar.update(idx+1)

        # Print and store epoch metrics
        print(loss.numpy(), r.result().numpy(), p.result().numpy())
        precision_history.append(p.result().numpy())
        recall_history.append(r.result().numpy())

        # Save checkpoints every 10 epochs
        if epoch % 10 == 0:
            checkpoint.save(file_prefix=checkpoint_prefix)

    # Plot precision and recall across epochs
    plt.plot(precision_history, label='Precision')
    plt.plot(recall_history, label='Recall')
    plt.xlabel('Epoch')
    plt.ylabel('Score')
    plt.legend()
    plt.title('Precision and Recall over Training')
    plt.show()
```

- GradientTape records math operations so we can backpropagate later
- X = set of two images, Y = label
- Calculate the loss using binary cross entropy
- Tape.gradient for backpropagation – how much each weight contributed to the loss
- apply_gradients uses those gradient calculations to update weights (gradient descent with Adam optimizer)

Training the Model

5.5 Train the Model

[69]:
```
EPOCHS = 20
```

[70]:
```
train(train_data, EPOCHS)
```

```
Epoch 1/20
27/27 ━━━━━━━━━━━━━━━━━━━━ 39s 1s/step

Epoch 2/20
```

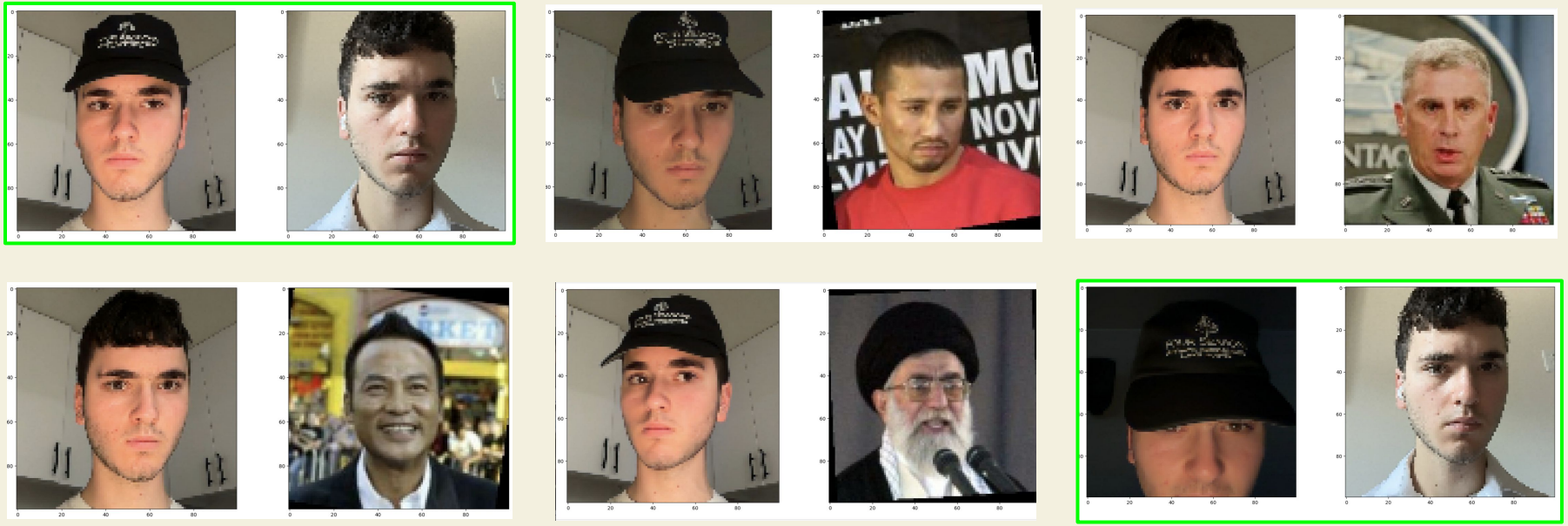
Making Predictions

- Precision – what proportion of positive identifications were actually correct
- Recall – what proportion of actual positives were identified correctly

```
[108]: # Make predictions
       y_hat = siamese_model.predict([test_input, test_val])
       y_hat

       1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 340ms/step

[108]: array([[9.9999851e-01],
              [8.8783645e-06],
              [9.3679937e-06],
              [3.7897642e-08],
              [1.9250213e-09],
              [9.9999428e-01],
              [1.1828179e-08],
              [8.3210878e-09],
              [2.6896123e-08],
              [9.9999994e-01],
              [9.7813825e-07],
              [9.9999869e-01],
              [9.9999869e-01],
              [9.9999905e-01],
              [9.9999666e-01],
              [3.5242387e-08]], dtype=float32)

[109]: # Post processing the results
       # If y_hat is > 0.5, return 1 else return 0
       [1 if prediction > 0.5 else 0 for prediction in y_hat]

[109]: [1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0]

[110]: y_true

[110]: array([1., 0., 0., 0., 0., 1., 0., 0., 0., 1., 0., 1., 1., 1., 1., 0.],
             dtype=float32)
```

```
[111]: # Creating a metric object
       m = Recall()

       # Calculating the recall value
       m.update_state(y_true, y_hat)

       # Return Recall Result
       m.result().numpy()

[111]: np.float32(1.0)

[112]: # Creating a precision object
       m = Precision()

       # Calculating the recall value
       m.update_state(y_true, y_hat)

       # Return Precision Result
       m.result().numpy()

[112]: np.float32(1.0)
```

Making Predictions

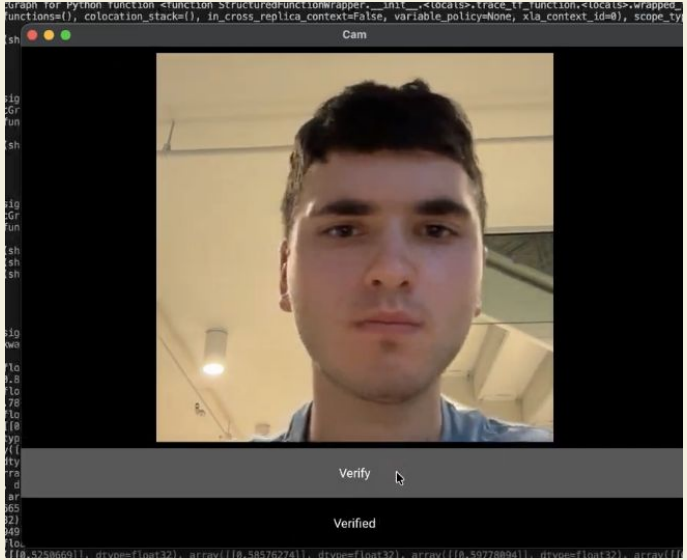


App Verification Logic

- Detection - how many of the 50 images scored above detection threshold
- Verification - proportion of images that scored above threshold
- Verified - true if proportion is greater than verification threshold meaning we have a match
- Comparing real-time webcam photo and checking against all the verification/reference images

```
# Grabbing results and wrapping it in NumPy array
# Summing up all examples that surpass detection_threshold
# Detection threshold - determines how many positive predictions are passing detection_threshold
detection = np.sum(np.array(results) > detection_threshold)
# Verification threshold - proportion of positive predictions / total positive samples
# What proportion of verification images matched input face?
verification = detection / len(results)
# If proportion is greater than verification threshold, person in webcam right now is verified
verified = verification > verification_threshold

# Set verification text
self.verification_label.text = 'Verified' if verified else 'Unverified'
```

App Demo



Results

```
 array([[0.75338286]
244]], dtype=float32
t32), array([[0.7631
7669868]], dtype=flo
=float32), array([[0
([[0.54328597]], dty
 dtype=float32), arr
rray([[0.7748608]],
]], dtype=float32),
[INFO   ] 108
[INFO   ] 1.0
[INFO   ] True
[INFO   ] 108
[INFO   ] 108
[INFO   ] 108
[INFO   ] 1
```

```
    # Log out details
    Logger.info(results)
    # Number of results that surpass detection threshold of 0.5
    Logger.info(detection)
    # What proportion of verification images matched input face?
    Logger.info(verification)
    # If proportion is greater than verification threshold, person in webcam right now is verified
    Logger.info(verified)

    # Number of results that surpass detection threshold
    Logger.info(np.sum(np.array(results) > 0.2))
    Logger.info(np.sum(np.array(results) > 0.4))
    Logger.info(np.sum(np.array(results) > 0.5))
    Logger.info(np.sum(np.array(results) > 0.8))

    return results, verified
```

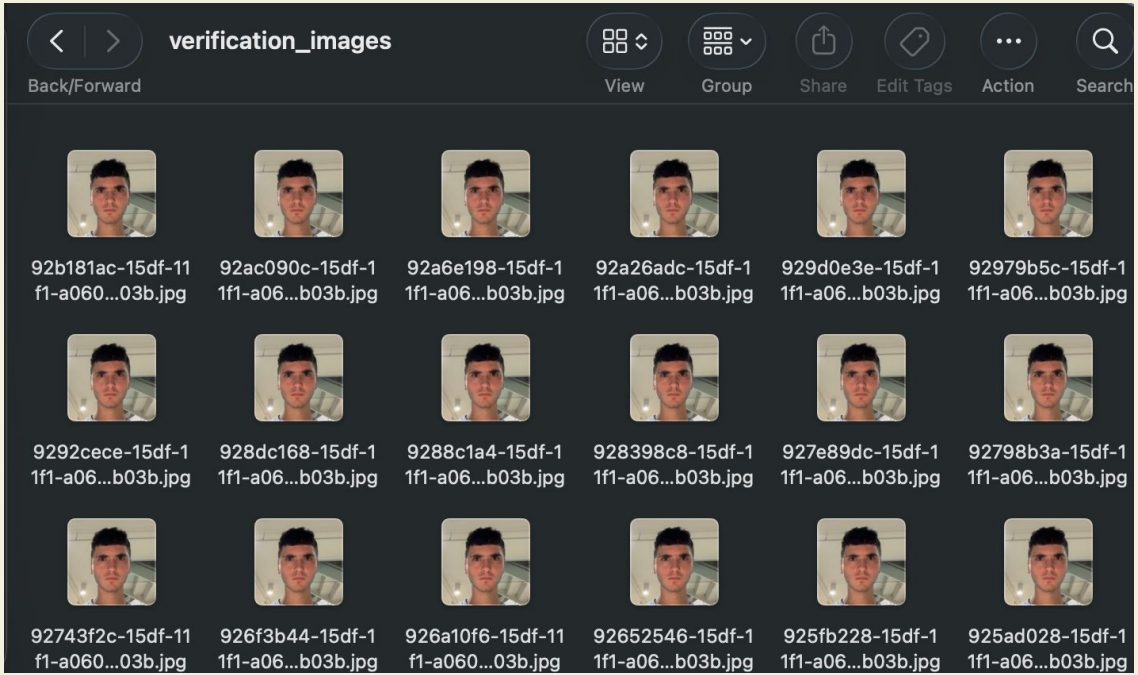
App Demo

ERROR!!

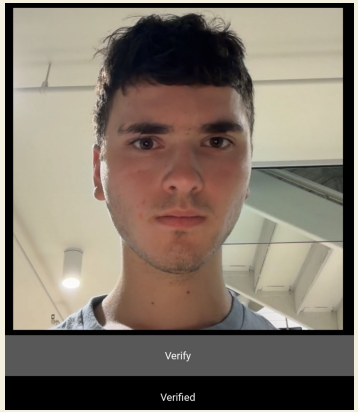
Either the model is wrong or I look like Adam Levine!

Improvements

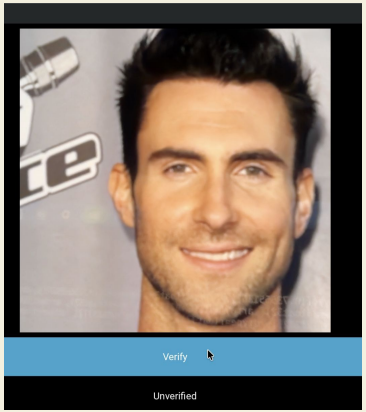
Added 300 Verification Images



Results!




```
rray([[0.83699995]], dtype=flo
type=float32), array([[0.67980
[[0.8734777]], dtype=float32),
=float32), array([[0.8754433]]
[INFO   ] 398
[INFO   ] 0.9754901960784313
[INFO   ] True
[INFO   ] 408
[INFO   ] 403
[INFO   ] 398
[INFO   ] 292
MIN: 0.20461068
MAX: 0.95760024
MEAN: 0.8164981
```




```
, dtype=float32), array([[0.08706767]]
array([[0.2165458]], dtype=float32), a
, dtype=float32), array([[0.05350673]]
[INFO   ] 61
[INFO   ] 0.14950980392156862
[INFO   ] False
[INFO   ] 316
[INFO   ] 127
[INFO   ] 61
[INFO   ] 1
MIN: 0.008724295
MAX: 0.83410364
MEAN: 0.3158892
```

```
)]
[INFO   ] 0
[INFO   ] 0.0
[INFO   ] False
[INFO   ] 0
[INFO   ] 0
[INFO   ] 0
[INFO   ] 0
MIN: 0.00012711769
MAX: 0.034080777
MEAN: 0.0020581335
```

Thank you!

Source Code

https://github.com/maxweinstein77/SiameseNetworkFacialRecognitionApp/tree/main

Works Cited

Koch, Gregory, et al. *Siamese Neural Networks for One-Shot Image Recognition*. 2015, www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf.

Li, Jessica. "Labelled Faces in the Wild (LFW) Dataset." Kaggle.com, 2018, www.kaggle.com/datasets/jessicali9530/lfw-dataset.

"Spotlight - Coded Bias Documentary." Ajl.org, 2016, www.ajl.org/facial-recognition-technology.

Youtube.com, 2026, www.youtube.com/watch?v=LKispFFQ5GU.